MODUL 5

Stack

Modul Praktikum Struktur Data • C++17

Program Studi Informatika • Praktikum Struktur Data • C++17

1. Tujuan Praktikum

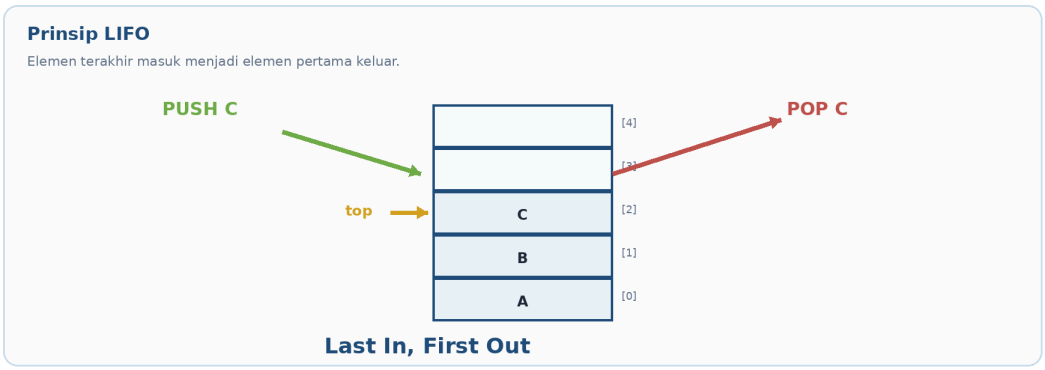
1. menjelaskan prinsip Last In, First Out (LIFO);
2. menjelaskan fungsi top pada stack;
3. membedakan stack kosong dan stack penuh;
4. mengimplementasikan stack menggunakan array;
5. melakukan operasi push, pop, dan peek;
6. menangani overflow dan underflow;
7. menampilkan isi stack tanpa mengubah keadaannya;
8. menggunakan stack untuk memeriksa pasangan tanda kurung; dan
9. menjelaskan implementasi alternatif menggunakan linked list.

2. Dasar Stack

Stack menyimpan data dengan aturan LIFO. Data yang terakhir dimasukkan menjadi data pertama yang dikeluarkan.

2.1 Prinsip LIFO

Penambahan dan pengambilan data dilakukan pada satu sisi yang disebut top.



Gambar 1. Prinsip LIFO pada stack.

Operasi	Keterangan
push	Memasukkan data ke bagian atas stack

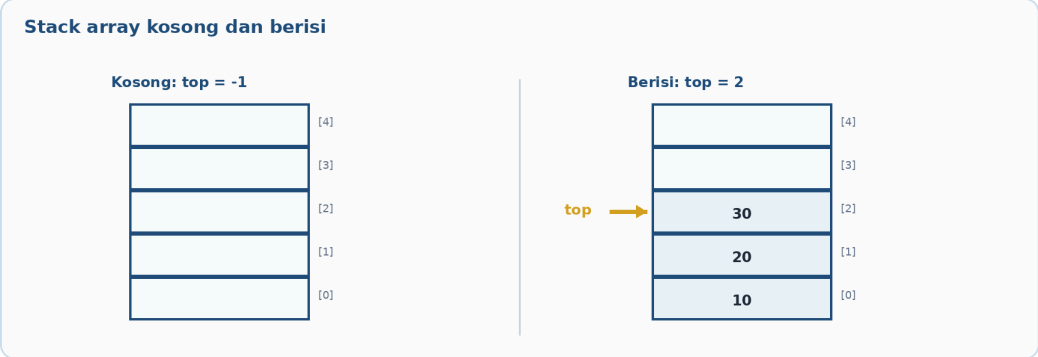
Operasi	Keterangan
pop	Mengambil dan menghapus data teratas
peek	Melihat data teratas tanpa menghapusnya
isEmpty	Memeriksa apakah stack kosong
isFull	Memeriksa apakah stack array penuh

2.2 Representasi menggunakan array

Array menyimpan elemen stack, sedangkan top menyimpan indeks elemen teratas.

```
const int KAPASITAS = 5;

struct Stack {
    int data[KAPASITAS];
    int top;
};
```



Gambar 2. Keadaan stack kosong, sebagian terisi, dan penuh.

2.3 Inisialisasi dan keadaan stack

Stack kosong ditandai dengan top == -1. Stack penuh terjadi ketika top mencapai indeks terakhir array.

```
void init(Stack& s) {
    s.top = -1;
}

bool isEmpty(const Stack& s) {
    return s.top == -1;
}

bool isFull(const Stack& s) {
    return s.top == KAPASITAS - 1;
}
```

2.4 Kondisi yang harus dijaga

- top berada pada rentang -1 sampai KAPASITAS - 1;
- elemen stack berada pada indeks 0 sampai top;
- push menaikkan top tepat satu posisi;
- pop menurunkan top tepat satu posisi;
- peek tidak mengubah top;

- push ditolak ketika stack penuh; dan
- pop serta peek ditolak ketika stack kosong.

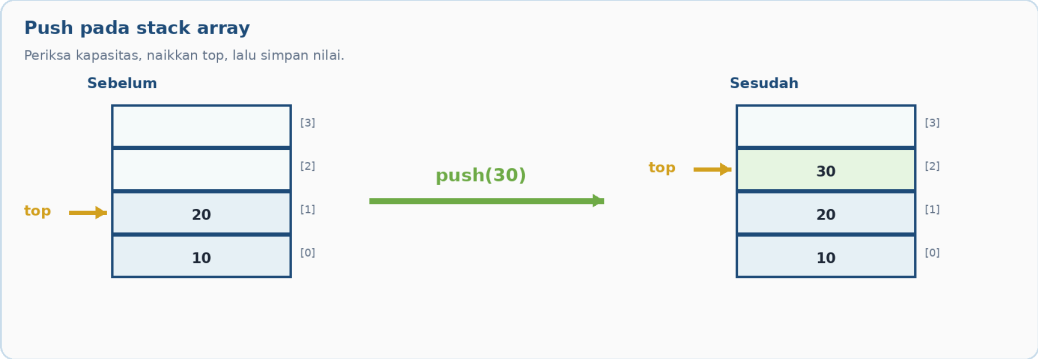
3. Operasi Stack

Operasi stack harus memeriksa keadaan penuh atau kosong sebelum mengubah top maupun mengakses array.

3.1 Push

Push memeriksa kapasitas, menaikkan top, kemudian menyimpan data.

```
bool push(Stack& s, int nilai) {
    if (isFull(s)) return false;
    ++s.top;
    s.data[s.top] = nilai;
    return true;
}
```

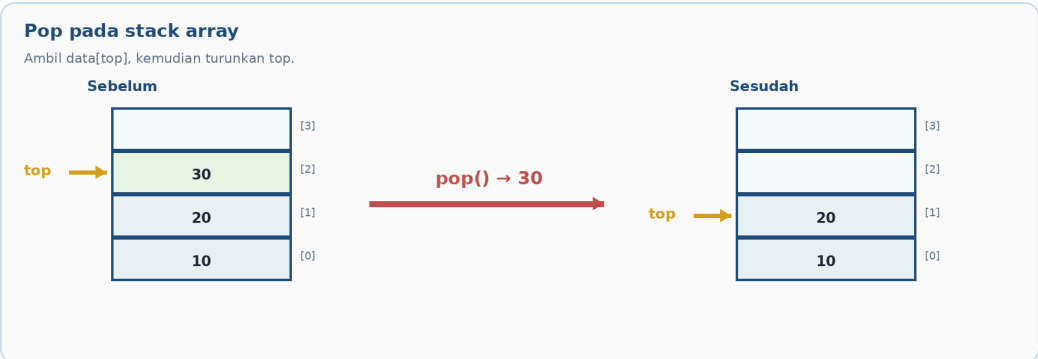


Gambar 3. Push menaikkan top sebelum data disimpan.

3.2 Pop

Pop mengambil data teratas kemudian menurunkan top. Nilai dikeluarkan melalui parameter reference.

```
bool pop(Stack& s, int& nilaiKeluar) {
    if (isEmpty(s)) return false;
    nilaiKeluar = s.data[s.top];
    --s.top;
    return true;
}
```



Gambar 4. Pop mengambil data teratas dan menurunkan top.

3.3 Peek

Peek membaca data teratas tanpa mengubah isi stack maupun top.

```
bool peek(const Stack& s, int& nilaiTeratas) {
    if (isEmpty(s)) return false;
    nilaiTeratas = s.data[s.top];
    return true;
}
```

3.4 Menampilkan dan mengosongkan stack

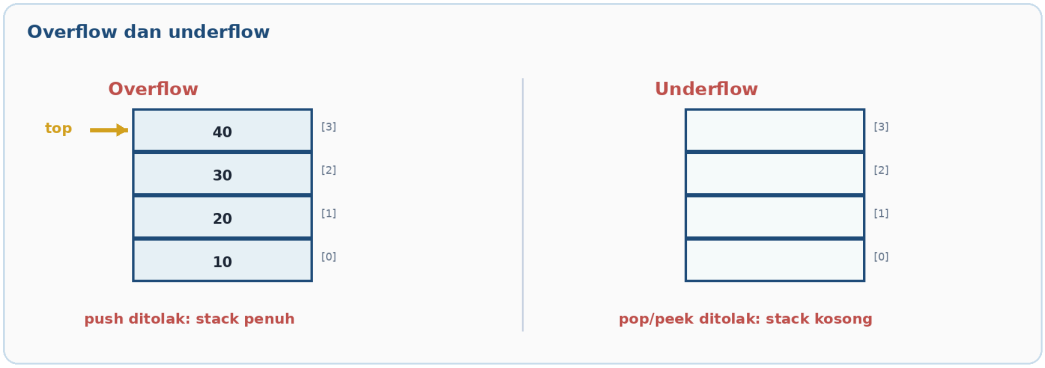
Penampilan menggunakan indeks bantu agar top tidak berubah. Clear cukup mengembalikan top ke -1.

```
void tampilkan(const Stack& s) {
    if (isEmpty(s)) {
        std::cout << "Stack kosong\n";
        return;
    }
    for (int i = s.top; i >= 0; --i)
        std::cout << s.data[i] << '\n';
}

void clear(Stack& s) {
    s.top = -1;
}
```

3.5 Overflow dan underflow

Overflow terjadi saat push dilakukan pada stack penuh. Underflow terjadi saat pop atau peek dilakukan pada stack kosong. Kedua keadaan ditangani sebelum array diakses atau top diubah.



Gambar 5. Overflow dan underflow harus ditolak tanpa mengubah stack.

3.6 Implementasi menggunakan linked list

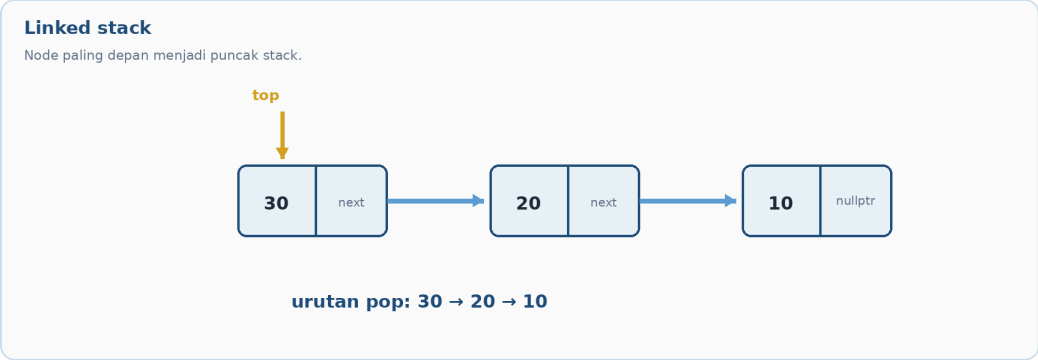
Stack juga dapat dibentuk dengan linked list. Aturan LIFO tetap sama, tetapi top berupa pointer dan setiap pop harus membebaskan node.

```
struct Node {
    int data;
    Node* next;
};

struct LinkedStack {
    Node* top;
};

bool push(LinkedStack& s, int nilai) {
    s.top = new Node{nilai, s.top};
    return true;
}

bool pop(LinkedStack& s, int& nilaiKeluar) {
    if (s.top == nullptr) return false;
    Node* hapus = s.top;
    nilaiKeluar = hapus->data;
    s.top = s.top->next;
    delete hapus;
    return true;
}
```



Gambar 6. Top pada linked stack menunjuk node teratas.

4. Kegiatan Praktikum

Gunakan satu berkas bernama latihan_stack.cpp. Tampilkan nilai top dan isi stack setelah setiap operasi.

4.1 Membentuk stack

Gunakan struktur Stack dan fungsi pada Bab 2.

1. buat objek Stack dan panggil init;
2. periksa isEmpty dan nilai top;
3. lakukan push terhadap 10, 20, dan 30;
4. tampilkan isi stack; dan
5. pastikan urutan tampil adalah 30, 20, 10.

4.2 Push dan kondisi penuh

Tambahkan fungsi push pada Subbab 3.1 ke dalam latihan_stack.cpp.

1. isi stack sampai penuh;
2. catat nilai top;
3. coba masukkan satu elemen tambahan;
4. pastikan push mengembalikan false; dan
5. pastikan top dan isi stack tidak berubah.

4.3 Pop, peek, dan kondisi kosong

Tambahkan fungsi pop dan peek pada Subbab 3.2 dan 3.3.

1. gunakan peek dan pastikan top tidak berubah;
2. lakukan pop dan periksa nilai yang keluar;
3. lakukan pop sampai stack kosong;
4. coba pop dan peek pada stack kosong; dan
5. pastikan kedua operasi ditolak tanpa mengubah top.

5. Tugas Praktikum

Buat program pemeriksa pasangan tanda kurung menggunakan stack karakter. Seluruh program disimpan dalam modul5_tanda_kurung.cpp.

1. push dilakukan ketika karakter (, [, atau { ditemukan;
2. tanda tutup memerlukan pop dari stack;
3. tanda buka dan tanda tutup harus berpasangan;
4. tanda tutup pada stack kosong dinyatakan tidak valid;
5. overflow dinyatakan tidak valid; dan
6. setelah seluruh teks diperiksa, stack harus kosong.

5.1 Kerangka program

Salin kerangka berikut ke modul5_tanda_kurung.cpp. Definisi fungsi ditambahkan setelah main pada berkas yang sama.

```
#include <iostream>
#include <string>

const int KAPASITAS = 100;

struct StackChar {
    char data[KAPASITAS];
    int top;
};

void init(StackChar& s);
bool isEmpty(const StackChar& s);
bool isFull(const StackChar& s);
bool push(StackChar& s, char nilai);
bool pop(StackChar& s, char& nilaiKeluar);
bool pasanganCocok(char buka, char tutup);
bool tandaKurungValid(const std::string& teks);
```

```
int main() {
    std::string teks;
    std::getline(std::cin, teks);

    if (tandaKurungValid(teks))
        std::cout << "Valid\n";
    else
        std::cout << "Tidak valid\n";

    return 0;
}
```

5.2 Fungsi yang dilengkapi

Lengkapi operasi StackChar dengan pola yang sama seperti stack integer. Setelah itu, lengkapi fungsi pemeriksaan berikut.

```
bool pasanganCocok(char buka, char tutup) {
    return (buka == '(' && tutup == ')') ||
           (buka == '[' && tutup == ']') ||
           (buka == '{' && tutup == '}');
}

bool tandaKurungValid(const std::string& teks) {
    StackChar s;
    init(s);

    for (char c : teks) {
        if (c == '(' || c == '[' || c == '{') {
            // lakukan push dan tangani overflow
        } else if (c == ')' || c == ']' || c == '}') {
            // lakukan pop dan periksa pasangan
        }
    }

    return isEmpty(s);
}
```

6. Pengujian

Jalankan program dengan seluruh skenario berikut. Catat hasil aktual dan perbaiki program jika hasilnya tidak sesuai.

No.	Masukan/Skenario	Hasil yang diharapkan
1	Stack baru	top == -1
2	Push satu data	top == 0
3	Peek	Nilai diperoleh, top tetap
4	Push saat penuh	Ditolak
5	Pop saat kosong	Ditolak
6	(a+b)	Valid
7	{a+[b*c]}	Valid
8	(a+b]	Tidak valid

No.	Masukan/Skenario	Hasil yang diharapkan
9	((a+b)	Tidak valid
10	a+b)	Tidak valid
11	Teks tanpa tanda kurung	Valid
12	Tanda buka melebihi kapasitas	Tidak valid

6.1 Diagnosis kode

Tentukan kesalahan, akibat, dan perbaikan setiap potongan kode berikut.

```
// A: top diubah sebelum pemeriksaan penuh
++s.top;
if (s.top == KAPASITAS) return false;

// B: indeks -1 diakses sebelum pemeriksaan kosong
nilai = s.data[s.top];
if (s.top == -1) return false;

// C: peek mengubah stack
nilai = s.data[s.top--];

// D: penampilan merusak stack
while (s.top >= 0)
    std::cout << s.data[s.top--];
```